

# SE446: Big Data Engineering

## Week 10A: Apache Kafka — Distributed Messaging

Prof. Anis Koubaa

SE 446

Alfaisal University

[https://github.com/aniskoubaa/big\\_data\\_course](https://github.com/aniskoubaa/big_data_course)

Spring 2026

# Today's Agenda

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Learning Objectives

By the end of this session you will be able to:

- 1 Explain **why** batch processing alone is insufficient and what problem Kafka solves
- 2 Describe Kafka's architecture: brokers, topics, partitions, offsets, replication
- 3 Trace a single message's journey from producer to consumer
- 4 Understand consumer groups and **why** they exist
- 5 Publish and consume messages using CLI and Python
- 6 Connect Kafka concepts to the Spark Streaming pipeline we build next session

## Where This Fits in Our Spark Journey

Until now, all our Spark jobs assumed **data already exists** on disk. Today we learn the **ingestion layer** that feeds real-time data into Spark. Next session (10B), we connect them: Kafka → Spark Structured Streaming.

# The Big Data Journey — Where Are We?



## What We Built So Far

Store (HDFS) → Process (MR/Spark)  
→ Query (Hive/SQL) → Optimize + ML

## The Missing Piece

All of these assume data **already sits** on disk. What if it **arrives non-stop**?

# The Problem: Batch Is Too Late

*Consider these three scenarios. Could you solve them with a nightly Spark batch job?*

## Fraud Detection

Stolen card used  
at 2:03 AM.  
Batch job runs  
at 6:00 AM.

**4 hours of fraud.**

## Surge Pricing

Concert ends;  
10K requests.  
Hourly batch  
sees it at 3:00.

**Passengers left.**

## IoT Alert

Server room hits 90°C.  
Alert must fire  
at 2:03 PM.

**Not tomorrow.**

## The Core Distinction

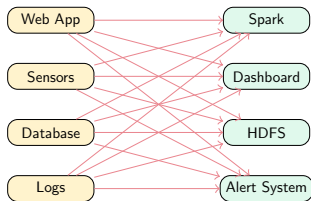
**Batch processing** answers: *“What happened?”*

**Real-time processing** answers: *“What is happening right now?”*

We need a system that can **receive events the instant they happen** and make them available to consumers **immediately**.

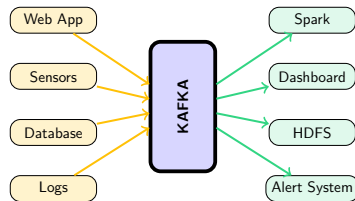
# The Integration Nightmare (Before Kafka)

*Imagine you have 4 source systems and 4 target systems. Without a central hub:*



$4 \times 4 = 16$  connections

Each with its own protocol, format, error handling



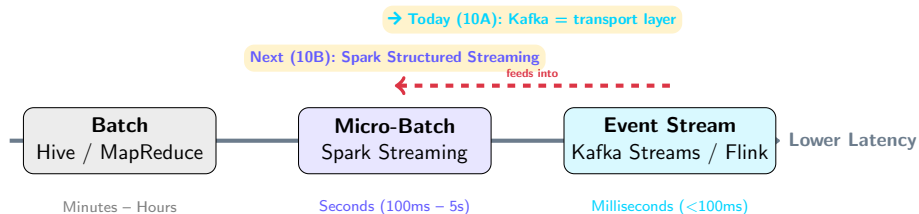
$4 + 4 = 8$  connections

One protocol, one format, one system

## Why This Matters

LinkedIn had this exact problem in 2011: dozens of systems needed to exchange data. Point-to-point connections became unmanageable. They built Kafka as a **central nervous system** — every system publishes to Kafka, every system reads from Kafka. **Producers and consumers never need to know about each other.**

# The Paradigm Spectrum — Where Kafka Sits



## How These Two Sessions Connect

**Kafka** = the highway that moves data between systems (today).

**Spark Structured Streaming** = the engine that processes data on that highway (next session).

In Session 10B, you will write: `spark.readStream.format("kafka")` — connecting directly to what you learn today.



# Kafka in One Sentence

*Imagine you are the CTO of a ride-sharing company.*

Every second, **100,000 events** happen: ride requests, GPS pings, payments, driver status changes.

You need a system that can:

- 🚀 **Receive** all of them without dropping any
- 💾 **Store** them reliably on disk (not just in memory)
- 👥 Let **10 different teams** read them independently, at their own pace
- ⚡ Do all of this at **millisecond latency**

**That system is Apache Kafka.**

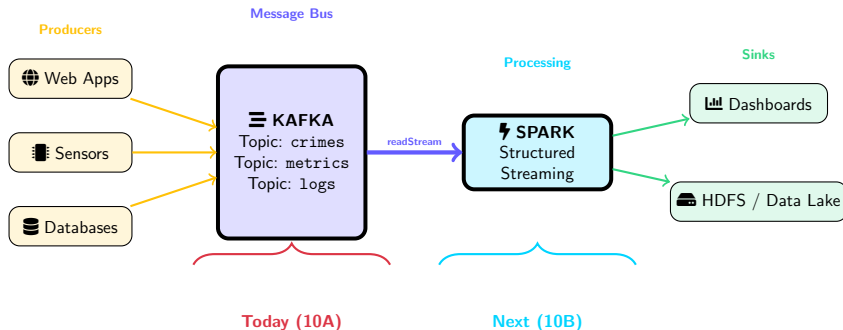
## Definition

Kafka is a **distributed, fault-tolerant, persistent event streaming platform**.

It acts as a **central hub** between data producers and data consumers.

Think of it as a **durable, replayable, high-speed message bus**.

# Where Kafka Fits in a Spark Architecture



## The Key Idea: Decoupling

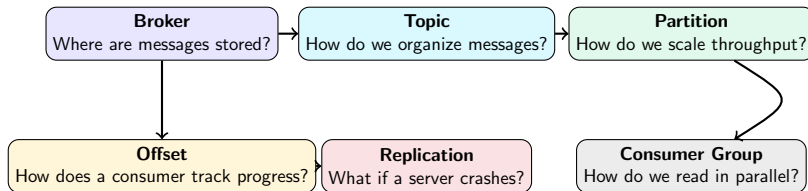
Producers and consumers are **completely independent**. A web app writes crime reports to Kafka. Spark reads them. Neither knows (or cares) about the other. You can add new producers or consumers at any time without changing anything.

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Building Kafka's Architecture Step by Step

*Instead of listing definitions, let us build the architecture from first principles.*  
For each concept, we ask: **what problem does it solve?**



## The Hierarchy

**Cluster**  $\supset$  **Broker(s)**  $\supset$  **Topic(s)**  $\supset$  **Partition(s)**  $\supset$  **Messages (with offsets)**

# Brokers and Topics

## Broker = A Kafka Server

- A single machine (or container) running Kafka
- Receives messages from producers
- Stores them on disk
- Serves them to consumers

### Why multiple brokers?

One machine has finite disk and bandwidth. A *cluster* of brokers distributes the load.

## Topic = A Named Category

- Like a database table or a folder
- Producers write to a topic by name
- Consumers subscribe to a topic by name

### Example topics for our crime data:

crimes — all crime reports

arrests — arrest events only

alerts — real-time alerts

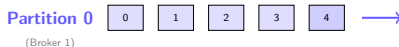
## ? Think About This

If a topic receives 1 million messages per second and lives on a single broker, what is the bottleneck? That single broker's disk I/O and network bandwidth. **This is why we need partitions.**

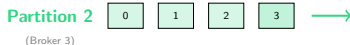
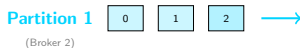
# Partitions — The Key to Kafka's Speed

A topic is split into **partitions** — independent, ordered, append-only logs.

Topic: “crimes” (3 partitions, spread across brokers)



Arrow = write position  
(append-only, immutable)



- ✓ Within a partition: **strict order**
- ✗ Across partitions: **no order guarantee**
- + More partitions = more parallelism

## Why Partitions Matter for Spark

Each Kafka partition maps to a **Spark task**.  
3 partitions = 3 parallel Spark tasks.  
This is how Spark achieves parallel reads.

## Design Consequence

You can **increase** partition count later.  
You **cannot decrease** it.  
Choose wisely at creation time.

# How Messages Are Assigned to Partitions

## No Key: Round-Robin

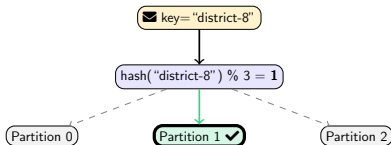
Messages spread evenly across all partitions.  
Good for **load balancing** when order doesn't matter.

```
# No key -> round-robin
producer.produce("crimes",
    value="theft at district 8")
```

## With Key: Hash Partitioning

$\text{hash}(\text{key}) \% \text{num\_partitions}$   
Same key  $\rightarrow$  same partition **always**.

```
# With key -> hash to partition
producer.produce("crimes",
    key="district-8",
    value="theft at district 8")
```

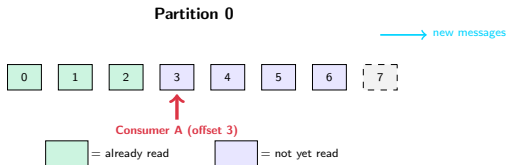


## ⚠ Watch Out: Hot Partitions

If you use district as the key and district 8 has 80% of all crimes, partition 1 gets 80% of the load. This is called a **hot partition** — the same skew problem we saw with Spark shuffles.

# Offsets — The Consumer's Bookmark

How does a consumer know where it left off? Every message gets a sequential **offset** within its partition.



## Starting Positions

earliest: Read from offset 0 (replay all)

latest: Read only new messages

## Retention

Kafka **retains** messages (default: 7 days).

Consumers can go back and re-read!

## ? What Happens If a Consumer Crashes?

Consumer A crashes after reading offset 2 but **before committing** offset 3. When it restarts, it re-reads from offset 2 — the last committed position. This is why Kafka's default delivery guarantee is **at-least-once**: safe, but you may see duplicates.



# Kafka Retains Messages — It Is Not a Queue

*This is the single most important difference between Kafka and traditional message queues.*

## Traditional Queue (RabbitMQ)

Consumer reads message → ACK → **deleted**  
No replay. No second reader.  
“Fire and forget”

## Kafka Log

Consumer reads message → ACK → **still there**  
Replay anytime. Multiple readers.  
“Replayable event log”

## Why Retention Matters — Three Concrete Scenarios

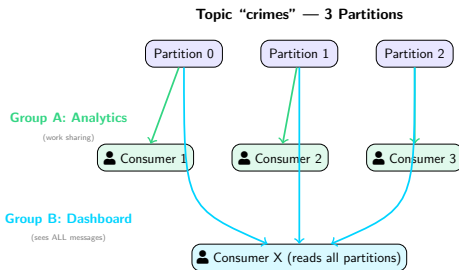
- 1 **New team joins:** The ML team wants to train on 30 days of crime data. They subscribe and read from earliest — no data pipeline needed.
- 2 **Bug fix replay:** Your Spark job had a bug. Fix it, reset the offset, **replay the same data** through the corrected pipeline.
- 3 **Multiple consumers:** The dashboard team AND the analytics team AND the alert system all read the **same messages** independently.

# Consumer Groups — Why Not Just One Consumer?

*Scenario: the crimes topic receives 50,000 messages/sec. One consumer processes 5,000 msgs/sec. What happens?*

**Consumer lag grows by 45,000 messages every second. You fall further behind forever.**

**Solution:** A *consumer group* — multiple consumers that **split the partitions** among themselves.



## Rule

1 partition → at most

Consumers > Partitions

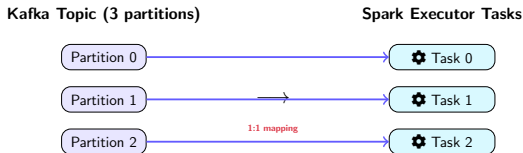
Extra consumers sit **idle**

Two Groups

Each group gets **all**

# Consumer Groups and Spark — The Connection

*When Spark reads from Kafka, what happens under the hood?*



## What Spark Does Automatically

- Creates a **consumer group** named after your Spark application
- Assigns **one Spark task per Kafka partition** (just like one consumer per partition)
- Tracks offsets using **checkpointing** (Spark's equivalent of committing offsets)

## 💡 Practical Implication

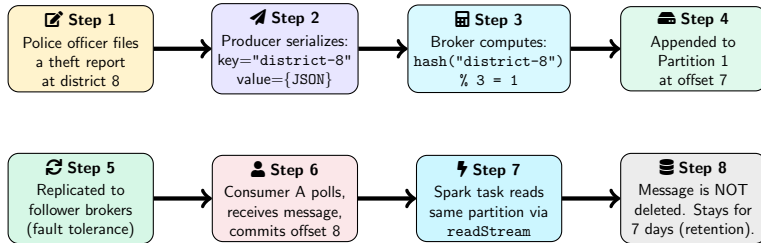
The number of Kafka partitions **directly controls** Spark's read parallelism. If you have 3 partitions but 10 Spark executors, 7 executors sit idle during the read phase.

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# A Crime Report's Journey Through Kafka

*Let us trace a single crime event from the moment it happens to the moment Spark processes it.*



## Key Observations

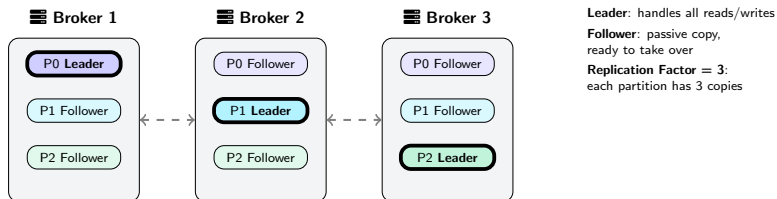
The message is **immutable** once written. It is **replicated** for safety. Multiple consumers (a Python script, a Spark job, a dashboard) can all read it **independently** without interfering with each other.

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?**
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Replication — Kafka's Safety Net

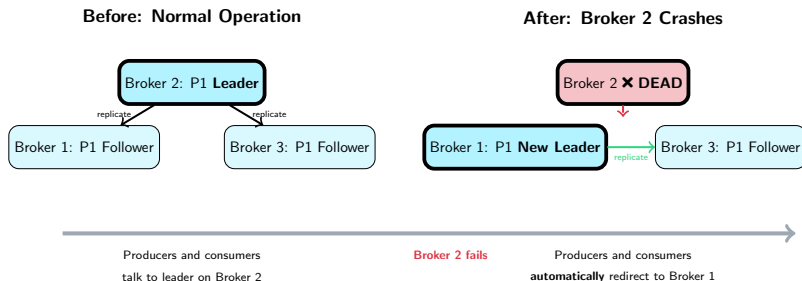
*What happens if the broker storing Partition 1 crashes? Without replication, all messages in that partition are **lost forever**.*



## The Rule

Producers write to the **leader** only. Followers replicate asynchronously. If the leader dies, a follower is **automatically elected** as the new leader. As long as at least one replica is alive, **no data is lost**.

# Broker Failure — What Actually Happens



## Why This Matters

- Producers and consumers **do not need to change their code**. Kafka handles failover.
- With `replication-factor=3`, Kafka tolerates **2 simultaneous broker failures**.
- This is why `--replication-factor` is a required parameter when creating topics.



# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues**
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Kafka vs RabbitMQ — Different Tools for Different Problems

| Feature             | Apache Kafka                     | RabbitMQ                   |
|---------------------|----------------------------------|----------------------------|
| <b>Mental model</b> | Append-only <b>log</b>           | Work <b>queue</b>          |
| Message after read  | ✓ <b>Retained</b> on disk        | ✗ <b>Deleted</b> after ACK |
| Replay              | Reset offset → re-read           | Not possible               |
| Throughput          | 1M+ msgs/sec per broker          | ~50K msgs/sec              |
| Ordering            | Guaranteed <b>per partition</b>  | Guaranteed per queue       |
| Multiple readers    | Independent consumer groups      | Message consumed once      |
| Best for            | Event streaming, log aggregation | Task distribution, RPC     |

## ⚠ Important Distinction

Kafka is **NOT** a replacement for RabbitMQ. Different tools for different problems.

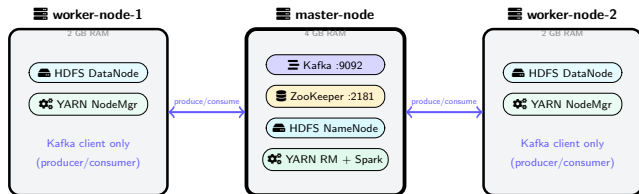
**Kafka** = “What happened in my system?” (event log, many readers)

**RabbitMQ** = “Do this job for me” (task queue, one worker picks it up)

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice**
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Our Cluster — Three Nodes, One Kafka Broker



## Connection Info

`bootstrap.servers = master-node:9092`  
Use this in all producers and consumers.

## Security

Port 9092 is **firewalled** — only cluster nodes and campus network can connect.  
ZooKeeper (2181) is localhost only.

# Our Cluster — Software Stack

| Component        | Version          | Role in Streaming Pipeline               |
|------------------|------------------|------------------------------------------|
| Hadoop / HDFS    | 3.4.1            | Distributed storage (data lake sink)     |
| YARN             | 3.4.1            | Resource management for Spark executors  |
| Spark            | 3.5.4            | Batch + streaming processing engine      |
| Hive             | 4.0.1            | SQL queries on stored data               |
| <b>Kafka</b>     | <b>3.9.0</b>     | <b>Real-time message ingestion (NEW)</b> |
| <b>ZooKeeper</b> | <b>(bundled)</b> | <b>Broker coordination (NEW)</b>         |
| Java             | OpenJDK 11       | Runtime for all services                 |
| Python           | 3.10+            | PySpark + confluent-kafka client         |

## The Full Pipeline You Are Building (M5)

**Python Producer**  $\xrightarrow{\text{port 9092}}$  **Kafka Topic**  $\xrightarrow{\text{readStream}}$  **Spark Streaming**  $\rightarrow$  **HDFS / Console**

Everything runs on this cluster. No external services needed.

# CLI Demo — Create, Produce, Consume

```
# Step 1: Create a topic with 3 partitions
kafka-topics.sh --create --topic demo-crimes \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# Step 2: List all topics
kafka-topics.sh --list --bootstrap-server localhost:9092

# Step 3: Start a consumer (Terminal 1 -- listening)
kafka-console-consumer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092

# Step 4: Produce messages (Terminal 2 -- typing)
kafka-console-producer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092
> {"type":"THEFT","district":8,"hour":14}
> {"type":"BATTERY","district":11,"hour":22}
```

## What You'll See

As soon as you type a message in Terminal 2, it appears **instantly** in the consumer in Terminal 1. This is real-time messaging in action.

# Python Producer — Annotated

Each line has a purpose. Let us walk through them.

```
from confluent_kafka import Producer          # (1) Kafka Python client
import json, time

conf = {'bootstrap.servers': 'MASTER_IP:9092'} # (2) Broker address
producer = Producer(conf)                     # (3) Create producer instance

crimes = [
    {"id": 1, "type": "THEFT", "district": 8, "arrest": False},
    {"id": 2, "type": "BATTERY", "district": 11, "arrest": True},
    {"id": 3, "type": "ROBBERY", "district": 3, "arrest": False},
]

for crime in crimes:
    producer.produce(
        topic="crimes",                        # (4) Which topic
        key=str(crime["district"]),            # (5) Partition key
        value=json.dumps(crime).encode("utf-8") # (6) Payload as bytes
    )
    print(f"Produced: {crime['type']} in district {crime['district']}")
    time.sleep(1)

producer.flush()                               # (7) Block until ALL messages are delivered
```

## Why These Choices Matter

- (5) Key = district → all crimes from the same district go to the same partition → **ordering per district**.
- (6) Kafka stores bytes, not objects. You must serialize (and deserialize on the consumer side).
- (7) produce() is **async** (returns immediately). Without flush(), your program may exit before messages are sent.

# Python Consumer — Annotated

```
from confluent_kafka import Consumer
import json

conf = {
    'bootstrap.servers': 'MASTER_IP:9092',
    'group.id': 'crime-analytics-group', # (1) Consumer group name
    'auto.offset.reset': 'earliest'     # (2) Start position
}
consumer = Consumer(conf)
consumer.subscribe(['crimes'])          # (3) Which topic(s) to read

try:
    while True:
        msg = consumer.poll(timeout=1.0) # (4) Pull model: consumer asks
        if msg is None:                  # for messages (not pushed)
            continue
        if msg.error():
            print(f"Error: {msg.error()}")
            continue
        crime = json.loads(msg.value().decode("utf-8")) # (5) Deserialize
        print(f"Type: {crime['type']} | Partition {msg.partition()} "
              f"| Offset {msg.offset()}")
finally:
    consumer.close()                    # (6) Cleanly leave the group
```

## Key Design Decisions

- (1) group.id: consumers with the same group ID share partitions. Different group ID = independent copy of all messages.
- (2) earliest = replay from the beginning. latest = only new messages.
- (4) poll(): Kafka uses a **pull model** — the consumer controls its own pace. This prevents slow consumers from being overwhelmed.



# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions**
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Delivery Guarantees — What Can Go Wrong?

When sending a message from producer to consumer, three things can go wrong: loss, duplication, or both.

| Guarantee                                    | Meaning                     | Kafka Config              | Scenario       |
|----------------------------------------------|-----------------------------|---------------------------|----------------|
| <b>At most once</b><br>(loss is OK)          | May lose, never duplicate   | acks=0                    | Log metrics    |
| <b>At least once</b><br>(duplicates handled) | Never lose, may duplicate   | acks=all                  | Crime reports  |
| <b>Exactly once</b><br>(must be exact)       | Neither lost nor duplicated | Idempotent + transactions | Bank transfers |



## Practical Advice

Most production systems use **at-least-once** with **idempotent consumers** (detecting and ignoring duplicates). Exactly-once adds significant overhead. **Spark Structured Streaming + Kafka checkpointing achieves exactly-once end-to-end** — covered in Session 10B.

# How Many Partitions? — A Design Checklist

*This is one of the first decisions you make when creating a topic. How do you choose?*

| Factor                | Recommendation                                                                                                  |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|
| Consumer parallelism  | At least as many partitions as max consumers in a group. <b>Also: at least as many as Spark executor cores.</b> |
| Expected throughput   | More partitions = higher throughput (each = sequential I/O)                                                     |
| Ordering requirements | Fewer partitions if strict global order matters. <b>1 partition = total order (but no parallelism).</b>         |
| Key cardinality       | If using keys, ensure enough distinct keys to spread evenly                                                     |
| Cluster limits        | Don't exceed ~10K partitions per broker                                                                         |

## 💡 Rule of Thumb

Start with the **number of consumers (or Spark cores) you expect**, then **double it** for future scaling. You can increase partitions later, but you **cannot decrease** them.

# Retention Policy — How Long Does Kafka Keep Messages?

```
# Time-based: keep messages for 7 days (default)
log.retention.hours=168

# Size-based: keep up to 1 GB per partition
log.retention.bytes=1073741824

# Compaction: keep only latest value per key
log.cleanup.policy=compact
```

## Time-Based (Most Common)

Messages older than `retention.hours` are deleted.  
Configurable per topic; 7 days is the default.

## Compaction (Advanced)

Only keeps the **latest value** per key.  
Useful for “current state” lookups  
(e.g., latest status of each district).

## Why This Matters for Your Spark Pipeline

If your Spark job goes down for 3 days and Kafka retains data for 7 days, you can restart and **replay everything you missed**.  
If retention is only 1 day, those 2 extra days of data are **gone**.

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World**
- 9 Summary and What's Next

# Who Uses Kafka and Why?

| Company   | Use Case                       | Scale                |
|-----------|--------------------------------|----------------------|
| LinkedIn  | Activity tracking, metrics     | 7 trillion msgs/day  |
| Netflix   | Real-time recommendations      | 1 PB/day             |
| Uber      | Trip events, pricing, matching | Millions of msgs/sec |
| Twitter/X | Stream processing, analytics   | Global event bus     |

## Why Kafka Won

No other system combined these four properties at the time:

High Throughput  
1M+ msgs/sec

Fault Tolerance  
replication

Persistence  
disk-backed log

Decoupling  
pub/sub model

## 💡 Fun Fact

Kafka was created at **LinkedIn** in 2011, open-sourced via Apache. Named after the author Franz Kafka — because it was designed for **writing** (data).

# Outline

- 1 Why Do We Need Kafka?
- 2 Kafka Architecture — The Why Behind Every Design Choice
- 3 Follow One Message — End to End
- 4 What If a Broker Dies?
- 5 Kafka vs Traditional Message Queues
- 6 Kafka in Practice
- 7 Kafka Design Decisions
- 8 Kafka in the Real World
- 9 Summary and What's Next

# Key Takeaways

- 1 **Why Kafka?** The  $N \times N$  integration problem + batch latency. Kafka provides a **central, persistent, high-throughput message bus**.
- 2 **Architecture:** Cluster  $\rightarrow$  Broker  $\rightarrow$  Topic  $\rightarrow$  Partition  $\rightarrow$  Offset. Messages are **ordered within a partition**, retained on disk, replicated for safety.
- 3 **Partitions** are the unit of parallelism — both for Kafka consumers **and** for Spark tasks that read from Kafka.
- 4 **Consumer Groups** enable parallel reading (work sharing within a group) and independent reading (different groups = pub/sub).
- 5 **Not a queue:** Messages are **retained, not deleted**. This enables replay, multiple consumers, and crash recovery.
- 6 **Fault tolerance:** Replication across brokers ensures no data loss even when machines fail.



## Core Formula

**Producers  $\rightarrow$  Kafka (Topics/Partitions)  $\rightarrow$  Consumer Groups**



# Sneak Peek — Connecting Kafka to Spark (Session 10B)

*Today we built the highway. Next session, we build the cars that drive on it.*

This is all it takes to connect Spark to the Kafka topics you created today:

```
# Read a stream of crime reports from Kafka
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "MASTER_IP:9092") \
    .option("subscribe", "crimes") \
    .load()
```

## What Happens Under the Hood

- Spark creates a **consumer group** and assigns one task per partition
- Each Kafka message arrives as a row with columns: key, value, topic, partition, offset, timestamp
- You parse value (bytes) into a structured DataFrame — same API you already know
- Spark tracks offsets via **checkpointing** — if it crashes, it resumes exactly where it left off

# What's Next

## Session 10B: Spark Structured Streaming

- The “infinite table” model — process streams with the DataFrame API you already know
- `readStream` from Kafka → parse JSON → aggregate → `writeStream`
- Output modes: append, update, complete
- Time windows: tumbling and sliding
- Watermarks for handling late data

## Today's Lab: Kafka Fundamentals

- Create topics, produce crime messages with Python
- Observe partition assignment with and without keys
- Experiment with consumer groups (1, 2, 3 consumers)
- Monitor consumer lag

## Milestone M5 Reminder

Due Week 12: End-to-end streaming pipeline (Kafka → Spark Streaming). Today you build the Kafka foundation; next session you complete the pipeline.